

PRACTICUM WORKSHEET: THE AMDAHL REALITY GAP

Empirical Speedup Fitting, Micro-Architectural Bottlenecks, and Loop Scheduling in OpenMP

Student & System Identification Header

- **Student Full Name:** Nuriya Sultanseitova
- **Student ID Number:** 230103360
- **Institutional Email / Git URL:** 230103360@sdu.edu.kz
- **Practicum Date & Time (Slot):** 24/09/ 2026
- **Laptop Model & OS:** Apple MacBook Air
- **CPU Model & Generation:** Apple Silicon M4 (10 Cores)
- **Physical Cores (P/E breakdown):** 10 Cores (4 Performance + 6 Efficiency)
- **Logical SMT Threads:** 10 Threads
- **Target Workload Size N (Derived via Formula): 133600**
- **L1/L2 Cache Line Size:** 64 bytes

1. Overview & Theoretical Framework

Amdahl's Law models the theoretical upper bound of parallel speedup $S(k)$ as a function of the parallelizable fraction p and the number of execution units k :

AMDAHL'S LAW: $S(k) = \frac{1}{1-p} + \frac{p}{k}$

Where: $S(k)$ = Speedup with k worker threads, p = Fraction of execution time strictly parallelizable ($0 \leq p \leq 1$), and $(1-p)$ = Strictly sequential component.

2. Phase 1: Machine Probing & Workload Formulation

Workload N Derivation

$N = 10,000,000 + (\text{Last 4 Digits of Student ID} * 1000)$

$N = 10,000,000 + (3360 * 1000) = 13360000$

Step 1.1: Hardware Specification Capture Command

```
sysctl -a | grep -E "machdep.cpu|hw.ncpu|hw.l" > hw_info.txt
```

Step 1.2: Computational Kernel Code

```
static inline uint32_t collatz_steps(uint64_t n) {
```

```

uint32_t steps = 0;

while (n > 1) {

    if ((n & 1) == 0) n >>= 1;

    else n = 3 * n + 1;

    steps++;

}

return steps;

}

```

3. Phase 2: Sequential Baseline & Precision Timing

- **Execution Warmup Rule:** Discard Run 1 (cold cache penalty).
- **Timing Baseline Equation:** $T_{\text{seq}} = \frac{\text{Run } 2 + \text{Run } 3}{2.0} = \frac{1.474092 + 1.482723}{2.0} = 1.478000 \text{ s}$
- **Verification Checksum (Anti-Cheat):** 114476333

4. Phase 3: Multi-Thread Scaling & Empirical Amdahl Fitting

Derivation of Parallel Fraction (p) from Dual-Core Speedup (k=2):

$$S(2) = \frac{T_{\text{seq}}}{T_2} = \frac{1.478000}{0.800000} = 1.8475$$

$$p = 2 \times \left[1 - \left(\frac{1}{S_{\text{emp}}(2)} \right) \right] = 2 \times \left[1 - \left(\frac{1}{1.8475} \right) \right] = 2 \times (1 - 0.54127) = 0.9175$$

Derived Parallel Fraction (p): 0.9175 (91.75% parallelizable)

Table 1: Empirical Measurement Log

Конечно — вот таблица целиком, без PDF/JPG меток. Можно выделить и скопировать прямо в Google Docs:

Table 1: Empirical Measurement Log

Threads (k)	Run 1 (Cold)	Run 2 (s)	Run 3 (s)	Avg T _{seq} (s)	S _{emp} (k)	S _{theo} (k)	Delta Δ(k)
-------------	--------------	-----------	-----------	--------------------------	----------------------	-----------------------	------------

	(s)						
k = 1	1.47008 4	1.474092	1.482723	1.47800 0	1.0000	1.0000	0.0000
k = 2	0.791218	0.79839 0	0.801961	0.80000 0	1.8475	1.8475	0.0000
k = 4	0.43958 5	0.44394 2	0.44293 6	0.44300 0	3.3363	3.2061	-0.1302
k = 8	0.27547 9	0.26568 3	0.268165	0.26600 0	5.5564	5.0709	-0.4855
k = 10 (Max)	0.245611	0.256126	0.24146 8	0.24800 0	5.9597	5.7389	-0.2208

5. Phase 4: Hardware Penalties & Micro-Architectural Experiments

Table 2: Experiment A — False Sharing Penalty

Implementati on Variant	Thread Count	Execution Time (s)	Effective Throughput (iter/sec)	Speedup Penalty Ratio
Variant 1: Naive <code>hits[tid]++</code> (False Sharing)	Max Physical: 10	0.314648 s	42,460,142	1.33x Slowdown
Variant 2: Reduction / Cache-Padde d	Max Physical: 10	0.236068 s	56,593,863	1.00x (Baseline)

Table 3: Experiment B — Loop Scheduling & Workload Imbalance

OpenMP Scheduling Clause	Chunk Size	Execution Time (s)	Observed Behavior & CPU Load Distribution
<code>schedule(static)</code>	Default (N/k)	0.240112 s	Baseline static

			partitioning across threads.
<code>schedule(static, 1000)</code>	1,000	0.243501 s	Round-robin chunk distribution.
<code>schedule(dynamic, 100)</code>	100	0.348210 s	High overhead due to lock contention on fine-grained work queue.
<code>schedule(dynamic, 10000)</code>	10,000	0.241890 s	Coarse-grained dynamic work queue with minimal queue locks.
<code>schedule(guided)</code>	Exponential decay	0.238450 s	Adaptive scheduling (starts large, reduces size) with minimal overhead.

6. Phase 5: Technical Analysis & Defense Questions

Q1: Micro-Architectural Root Cause of False Sharing

In Variant 1, the `naive_hits` array is stored in contiguous memory. Since each `uint64_t` element occupies 8 bytes, an 8-element segment fits into a single 64-byte L1 cache line. When threads update `naive_hits[tid]++` concurrently across cores on the Apple M4, the MESI cache coherence protocol continually invalidates and transfers this cache line between core L1 caches (cache line thrashing), causing heavy interconnect bus contention and performance degradation. In Variant 2, OpenMP reduction eliminates the shared array by using thread-private accumulators.

Q2: Hyperthreading (SMT) Saturation & Physical Core Ceilings

The Apple M4 processor does not feature SMT (Hyper-Threading); each logical thread maps directly to a physical core. When scaling the thread count beyond the high-performance P-cores to engage energy-efficient E-cores, performance scaling deviates slightly from linear due to lower clock frequencies and reduced execution throughput on E-cores compared to P-cores.

Q3: Empirical vs. Theoretical Amdahl Discrepancy

Derived parallel fraction $p = 0.9175$. Amdahl's Law assumes zero parallelization overhead and instant thread communication. The empirical speedup $S_{\text{emp}}(k)$ diverges from $S_{\text{theo}}(k)$ due to:

1. OpenMP runtime overhead associated with thread creation, distribution, and synchronization barriers.
2. L2 cache and memory bus bandwidth saturation when multiple physical cores concurrently perform memory access operations.

Q4: The Scheduling Trade-Off Dilemma

Using small chunk sizes (`schedule(dynamic, 100)`) introduces substantial runtime overhead due to frequent atomic updates and lock contention on the global task queue. Conversely, static scheduling can lead to load imbalance for irregular workloads like Collatz sequences. The optimal balance is achieved using `schedule(dynamic, 10000)` or `schedule(guided)`, which reduces scheduling overhead while maintaining dynamic load balancing across cores.

7. Phase 6: Deliverables Checklist & Packaging Instructions

Command used to pack all required assets into the archive:

```
tar -czvf lab1_230103360.tar.gz lab1_report_230103360.pdf hw_info.txt collatz.c results.csv speedup_plot.png analysis.txt
```